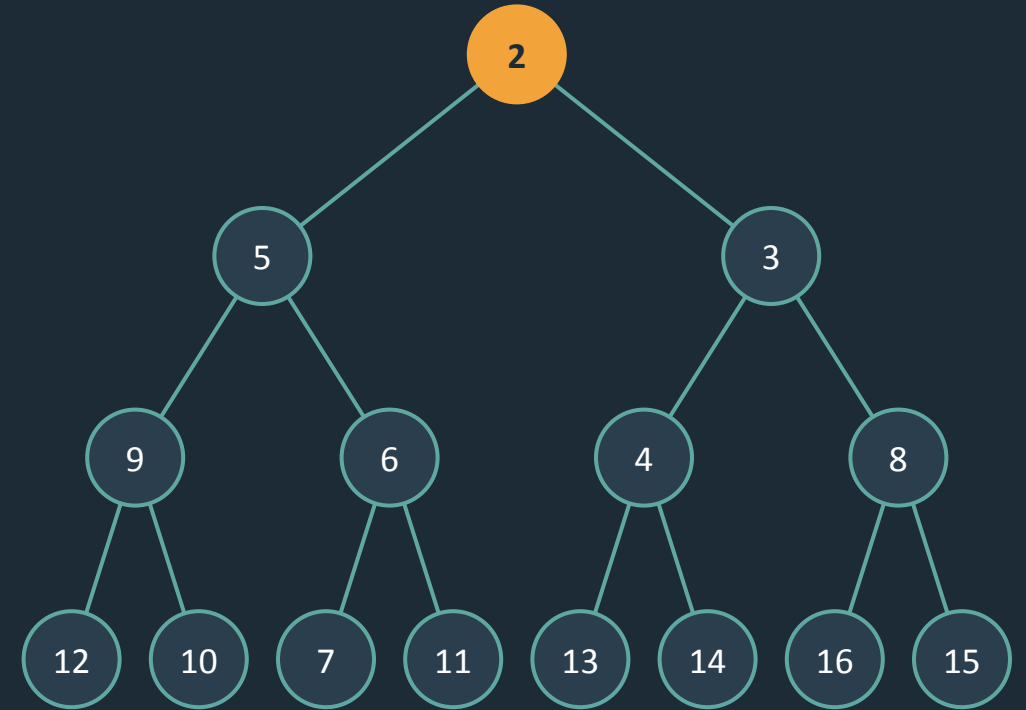


AIL 7002

Priority Queues, Heaps & HeapSort

From the ADT to binary, d-ary and binomial heaps — and
sorting in place

Lecture · 75 minutes



Today's plan

1

The Priority Queue ADT

Operations, keys and total orders, naive list implementations, PQ-Sort

10 min

2

Binary heaps

Two invariants, array layout, sift-up / sift-down, $O(n)$ bottom-up build

22 min

3

HeapSort

Sorting in place, analysis, and the $\Omega(n \log n)$ comparison lower bound

13 min

4

Mergeable heaps

d-ary heaps, binomial heaps, meld as binary addition, amortized cost

21 min

5

Applications & wrap-up

Dijkstra / Prim, heaps in standard libraries, takeaways, exercises

9 min



1

PART 1

The Priority Queue ADT

What we want to support — before deciding how

≈ 10 min

Beyond FIFO: serve the most urgent element next

A FIFO queue serves elements in arrival order. Many problems need to process elements according to a **priority** instead.

A **priority queue** is a collection that supports

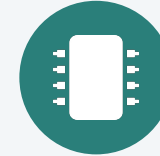
- insertion of arbitrary elements, and
- removal of the element with **first priority**.

The priority of an element is its **key**: any object whose instances can be compared to establish a natural ordering.

Convention today: **min**-oriented (smallest key = first priority).



ER triage



CPU / job scheduling



Event-driven simulation



Bandwidth management



Dijkstra, A* search



Prim's MST



Huffman coding



HeapSort, top-k, median

Highlighted: graph algorithms we revisit in Part 5.

The Priority Queue ADT

Core operations

insert(k, v)	create entry with key k, value v
min()	return (not remove) an entry with minimal key
removeMin()	remove and return it (null if empty)
size(), isEmpty()	

Extended (graph algorithms need these)

decreaseKey(e, k)	lower the key of entry e to $k \leq \text{key}(e)$.
delete(e)	remove entry e from the queue
meld(H₁, H₂)	replace H ₁ , H ₂ by their union

e is a **handle** to an entry already in the queue.

```
interface Entry<K, V> {
    K getKey();
    V getValue();
}
```

- Same operations, different names: removeMin = EXTRACT-MIN = deleteMin = poll; meld = UNION = merge.
- Duplicate keys are allowed. Max-oriented queues are symmetric: flip every comparison.

Keys must come from a total order

Comparison rule $\leq$ on keys must satisfy, for all x, y, z :

Reflexive $x \leq x$

Antisymmetric $x \leq y \wedge y \leq x \Rightarrow x = y$

Transitive $x \leq y \wedge y \leq z \Rightarrow x \leq z$

Total $x \leq y$ or $y \leq x$

Then comparisons never contradict each other, and "the minimum" is always well defined.

keys belong to a total order

2-D points, x first then y

$(3,4) \leq (3,5) \leq (4,3)$: every pair comparable $\rightarrow$ **total order** ✓

2-D points, x and y both

$(2,1) \leq (4,3)$, but $(4,3)$ vs $(3,4)$? Neither holds $\rightarrow$ only a **partial order** ✗

Design lesson: separate the comparator from the element type

```
priority_queue<Pt, vector<Pt>, LeftRight> pq;
new PriorityQueue<>(Comparator.comparingInt(p -> p.y));
```

Two naive implementations — and their trade-off

Unsorted list



insert

$O(1)$

min, removeMin

$O(n)$

Insert anywhere; must scan the whole list to find the smallest key.

Sorted list



insert

$O(n)$

min, removeMin

$O(1)$

Minimum sits at the head; insertion must find its place first.

Each makes one side cheap and the other expensive. Can we **balance** them? A heap makes both $O(\log n)$.

PQ-Sort: every priority queue is a sorting algorithm

PQ-SORT (S) :

```

P ← empty priority queue
while S not empty:           ▷ phase 1
    P.insert(S.removeFirst())
while P not empty:           ▷ phase 2
    S.insertLast(P.removeMin())
  
```

PQ implementation	Phase 1	Phase 2	Resulting sort
Unsorted list	$O(n)$	$O(n^2)$	Selection sort, $O(n^2)$
Sorted list	$O(n^2)$	$O(n)$	Insertion sort, $O(n^2)$
Heap	$O(n \log n)^*$	$O(n \log n)$	HeapSort, $O(n \log n)$

* If all input items are first placed in the heap array and BUILD-HEAP replaces the n individual insertions, Phase 1 becomes $O(n)$.

Where is the waste?

- A sorted list maintains the **complete** order of all elements at all times — but removeMin only ever needs the smallest one.
- Finding the minimum does not require ordering everything. A heap keeps just enough structure to know the minimum, and repairs that structure along a single path.



2

PART 2

Binary heaps

Two invariants, one array, two primitive moves

≈ 22 min

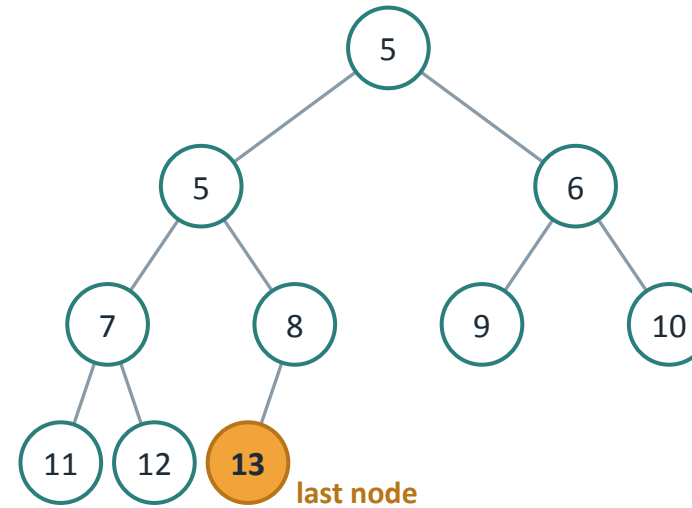
A heap is a binary tree with two invariants

① Heap order (relational)

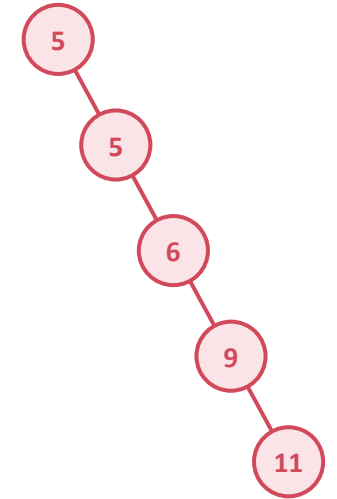
- For every node v other than the root: $\text{key}(v) \geq \text{key}(\text{parent}(v))$.
- Keys are non-decreasing along every root-to-leaf path.
- Hence the minimum key is at the root.

② Complete tree (structural)

- Levels $0, 1, \dots, h-1$ hold the maximum possible number of nodes.
- Nodes on level h sit in the leftmost positions.
- The **last node** is the rightmost node of maximum depth.



A heap: both invariants hold



Heap-ordered, but height $n-1$

Order alone is not enough: we must **avoid bad tree configurations**. Completeness pins the shape so that the height stays logarithmic.

A heap with n entries has height $h = \lfloor \log_2 n \rfloor$

depth	# nodes
0	1
1	2
2	4
$\vdots$	$\vdots$
$h - 1$	2^{h-1}
h	between 1 and 2^h

Levels 0 ... $h-1$ are full: $1 + 2 + \dots + 2^{h-1} = 2^h - 1$ nodes.

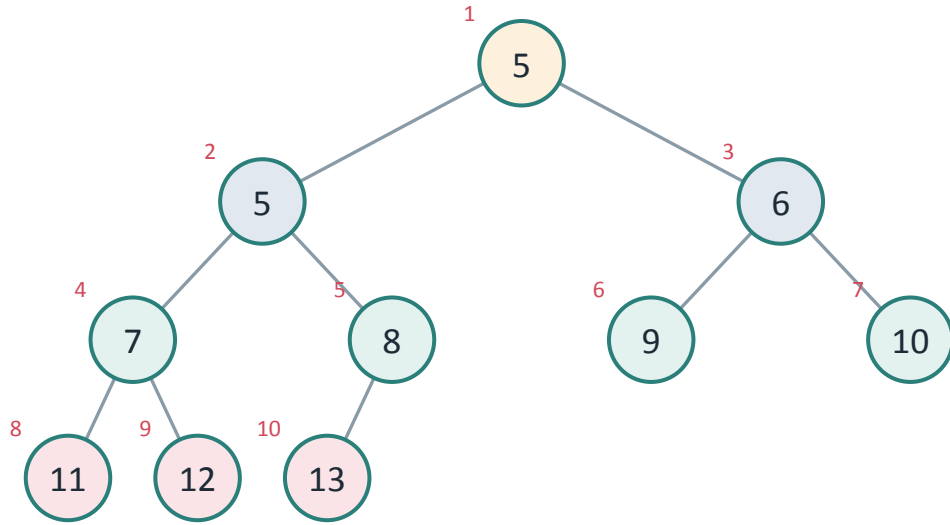
Level h holds at least 1 and at most 2^h nodes, so

$$2^h \leq n \leq 2^{h+1} - 1$$

$$\Rightarrow h \leq \log_2 n < h + 1 \Rightarrow h = \lfloor \log_2 n \rfloor$$

Consequence: if every PQ operation does work proportional to the height of the heap — i.e., walks one root-to-leaf path — it runs in **$O(\log n)$** time.

No pointers needed: store the heap in an array



Level order into $A[1..n]$; $A[n]$ is the last node, $A[n+1]$ the next free slot.

1-indexed (CLRS)

$\text{parent}(i) = \lfloor i / 2 \rfloor$ $i \gg 1$

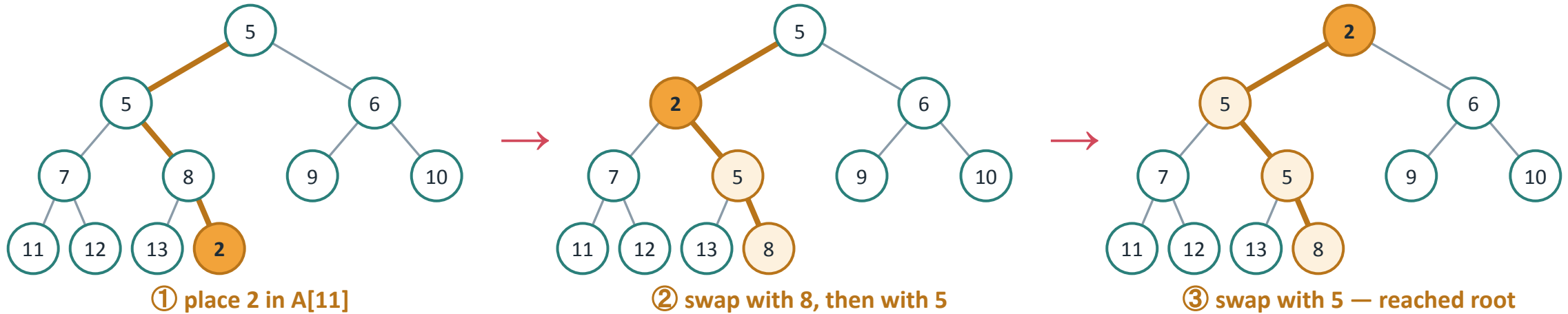
$\text{left}(i) = 2i$ $i \ll 1$

$\text{right}(i) = 2i + 1$ $(i \ll 1) \mid 1$

Heap property: $A[\text{parent}(i)] \leq A[i]$

- **0-indexed** (Python `heapq`, C++ STL): $\text{parent} = \lfloor (i-1)/2 \rfloor$, children $2i+1, 2i+2$.
- With only the usual parent/left/right pointers, locating the next level-order slot takes $O(\log n)$; the array identifies it immediately as $A[n+1]$

insert(k): add at the end, then sift up



INSERT(A, k) :

$n \leftarrow n + 1; \quad A[n] \leftarrow k$

SIFT-UP(A, n)

SIFT-UP(A, i) :

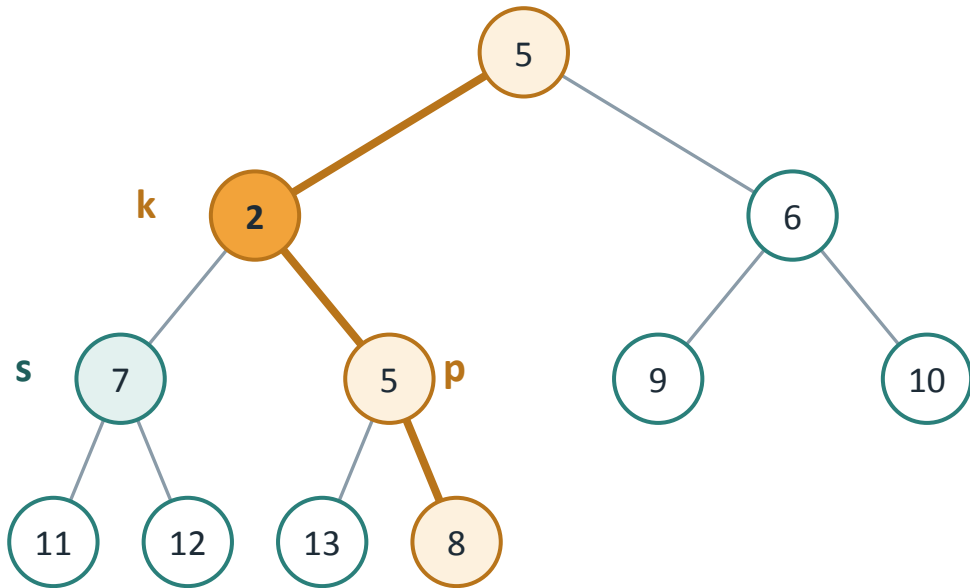
while $i > 1$ and $A[\text{parent}(i)] > A[i]$:

 swap $A[i] \leftrightarrow A[\text{parent}(i)]$

$i \leftarrow \text{parent}(i)$

- Keeps the tree complete (new node = next slot); repairs heap order along **one** leaf-to-root path.
- At most h swaps $\Rightarrow O(\log n)$.
- Also called upheap, swim, bubble-up, percolate-up.

Why sift-up is correct



Mid-way through insert(2): $k = 2$ has just swapped with its parent $p = 5$, which moved down. s is the sibling of the slot k came from.

Claim: only nodes on the path change.

- Invariant: heap order holds everywhere except possibly between k and its parent.
- When k swaps with its parent p , the old key of p moves **down** into the slot k came from. It was already an ancestor of everything below that slot, so it is $\leq$ all of them.
- The sibling s of that slot now has parent k , and $k < p \leq s$. So s is fine too.
- Each swap therefore changes only two adjacent nodes on the same root-to-leaf path.
- The loop stops at the root, or when $\text{parent}(k) \leq k$: the invariant then says the whole tree is a heap.

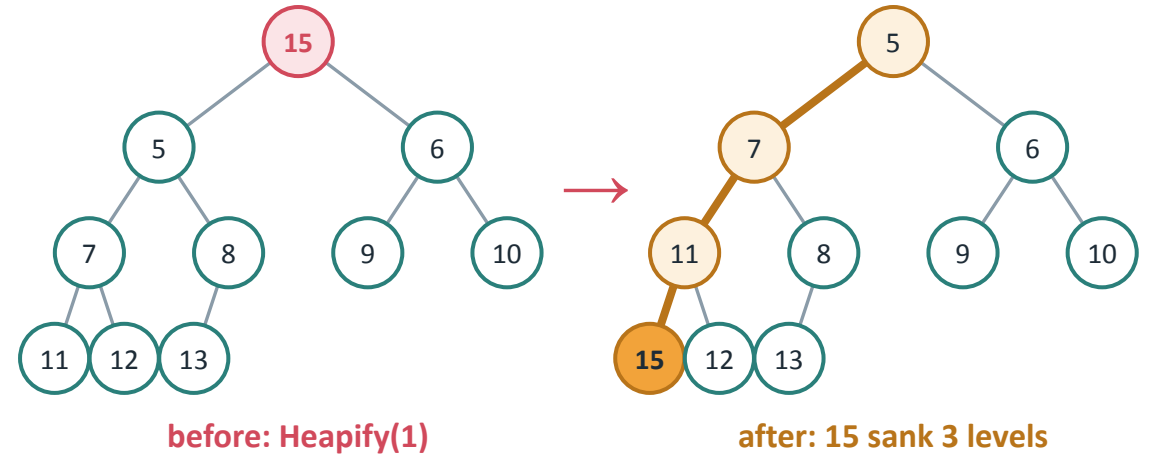
Heapify(i): sift an element down

Precondition: the subtrees rooted at $\text{left}(i)$ and $\text{right}(i)$ are heaps; $A[i]$ may violate heap order. Goal: make the subtree rooted at i a heap by moving $A[i]$ down.

```

HEAPIFY(A, i):                                ▷ sift-down
  l ← 2i;  r ← 2i + 1;  m ← i
  if l ≤ n and A[l] < A[m]:  m ← l
  if r ≤ n and A[r] < A[m]:  m ← r
  if m ≠ i:
    swap A[i] ↔ A[m]
    HEAPIFY(A, m)
  
```

- Swap with the **smaller** child: it becomes the parent of its sibling, so it must be $\leq$ it.
- Cost $O(\text{height of } i) = O(\log n)$ in the worst case. Also: downheap, sink, MIN-HEAPIFY.



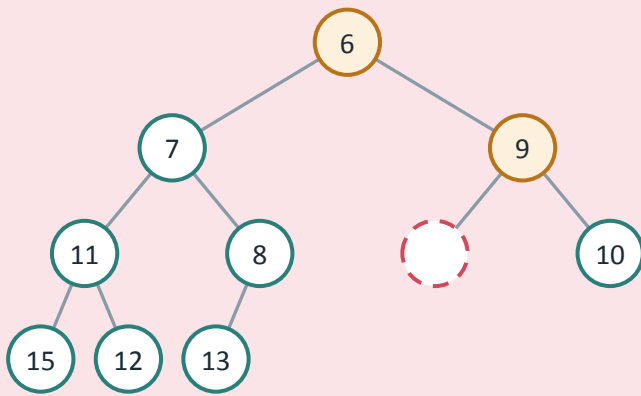
Why it works

- Elements on the path move up one level; each was the smaller child, so it is $\leq$ its sibling.
- $A[i]$ stops at the last node j on the path whose children are both $\geq A[i]$ (or at a leaf).

removeMin(): refill the root with the last element

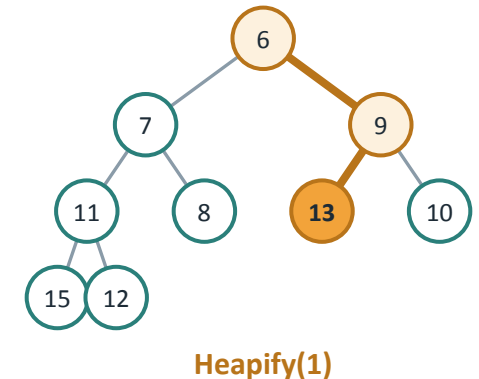
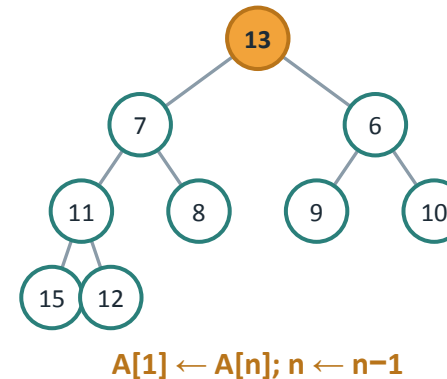
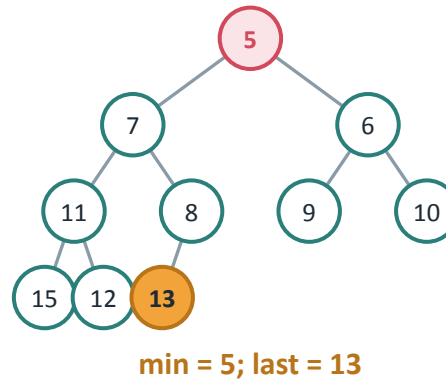
✗ First attempt

Delete the root and repeatedly promote the smaller child. The hole travels down...



...and ends at an arbitrary leaf: the tree is **no longer complete**.

✓ Idea: the heap loses one node anyway — so move the **last** element into the root and sift it down.

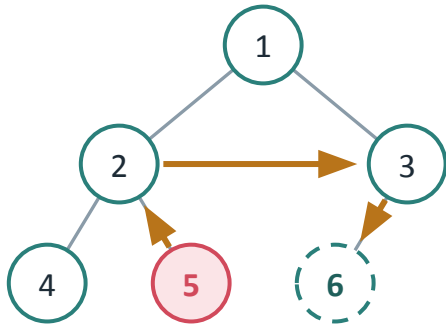


REMOVE-MIN (A) :

```
min ← A[1]
A[1] ← A[n];    n ← n - 1
HEAPIFY(A, 1)
return min
```

- Structure fixed first, order second.
- One root-to-leaf path $\Rightarrow O(\log n)$.
- min() alone: A[1], $O(1)$.

Linked heaps: finding the next slot in $O(\log n)$



$n = 5$: last $101_2 \rightarrow$ slot 110_2

up to a left child, across, down-left



last node

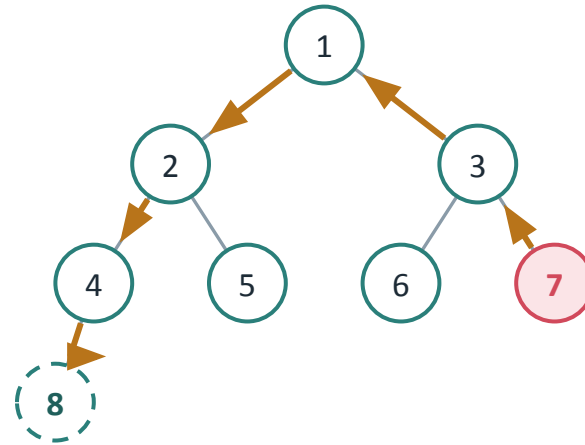


insertion slot



search path

labels = positions



$n = 7$: last $111_2 \rightarrow$ slot 1000_2

up to the root, then down-left (new level)

insert: find the next free slot (position $n + 1$)

- ① From the last node, go up until you reach a **left child** or the root.
- ② At a left child, move to its **right sibling**.
- ③ Go **down-left** to the first empty position.

removeMin: find the new last node (position $n - 1$)

- ① From the old last node, go up until you reach a **right child** or the root.
- ② At a right child, move to its **left sibling**.
- ③ Go **down-right** to the bottom level.

Each walk visits $\leq 2h + 1$ nodes $\Rightarrow O(\log n)$. An array gets both for free: $A[n+1]$ and $A[n-1]$.

Why it works: a binary counter

- Drop the leading 1 of position i in binary: the remaining bits are the root-to-node path (0 = left, 1 = right).
- $n \rightarrow n+1$: trailing 1s become 0s (climb past right children), the lowest 0 becomes 1 (step to the right sibling), then 0s follow (go down-left).

The remaining operations reuse the same two moves

decreaseKey(*i*, *k*)

$O(\log n)$

$A[i] \leftarrow k \ (k \leq A[i]); \text{ SIFT-UP}(A, i)$

increaseKey(*i*, *k*)

$O(\log n)$

$A[i] \leftarrow k \ (k \geq A[i]); \text{ HEAPIFY}(A, i)$

delete(*i*)

$O(\log n)$

$A[i] \leftarrow A[n]; n \leftarrow n - 1; \text{ then SIFT-UP or HEAPIFY at } i \text{ — whichever applies}$

meld(*H*₁, *H*₂)

$\Theta(n)$

No structural shortcut: concatenate the arrays and rebuild

Handles. Entries move around the array on every swap, so decreaseKey and delete need a position map entry $\rightarrow$ index, updated inside each swap (an **indexed priority queue**). Without it, finding the entry costs $O(n)$.

Building a heap from n elements

Top-down: n successive inserts

$\sum_{i=1..n} O(\log i) = \Theta(n \log n)$ in the worst case (e.g., keys arriving in decreasing order).

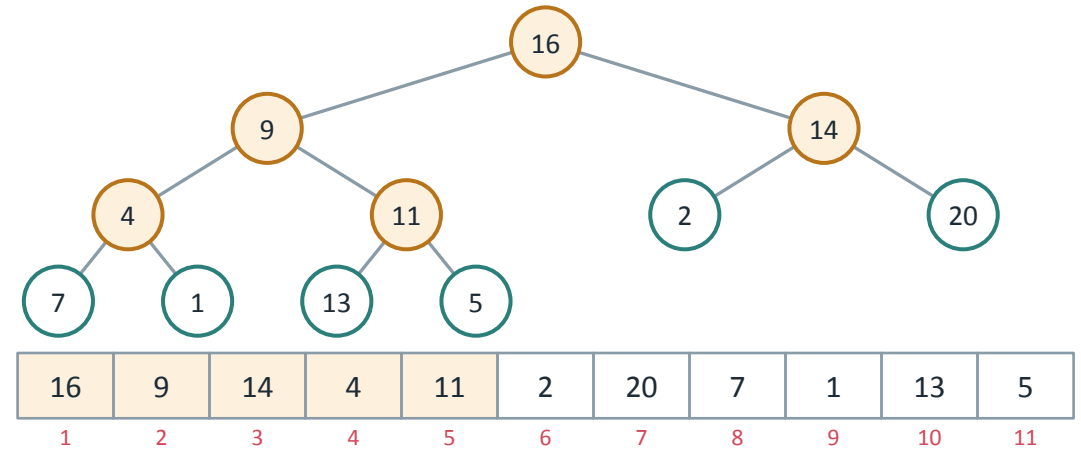
Bottom-up (Floyd, 1964)

Leaves ($i > \lfloor n/2 \rfloor$) are already heaps, so start there and fix subtrees bottom-up.

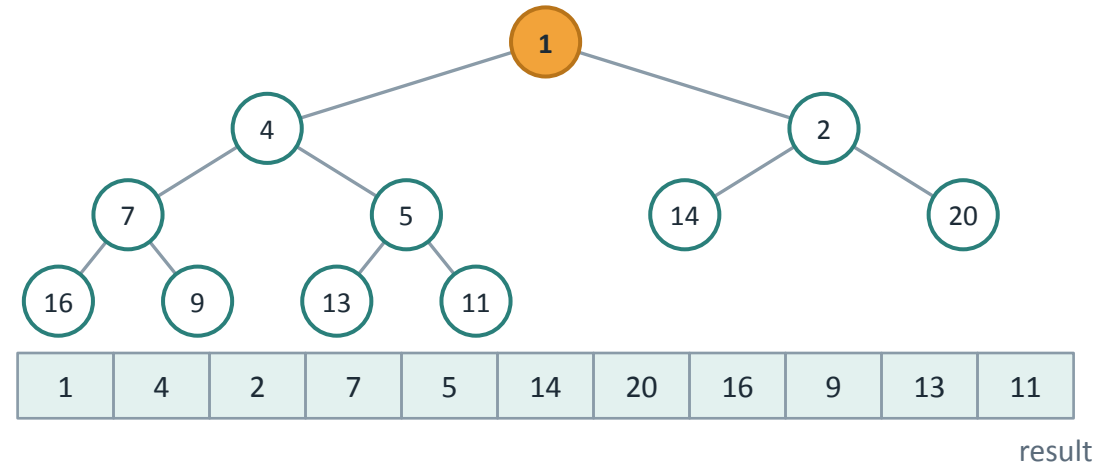
BUILD-HEAP (A) :

```
for i ← ⌊n/2⌋ downto 1:
    HEAPIFY(A, i)
```

Correctness (induction on i): before iteration i, every node $j > i$ roots a heap — so Heapify(i)'s precondition holds.



input: internal nodes 1..5 (shaded) get Heapify, in order 5, 4, 3, 2, 1



Bottom-up construction runs in $O(n)$

- Heapify(i) costs $O(\text{height of } i)$.
- At most $\lceil n / 2^{h+1} \rceil$ nodes have height h .
- Total $\leq \sum_{h=0.. \lfloor \log n \rfloor} \lceil n / 2^{h+1} \rceil \cdot O(h) = O(n \cdot \sum_{h \geq 0} h / 2^h)$.

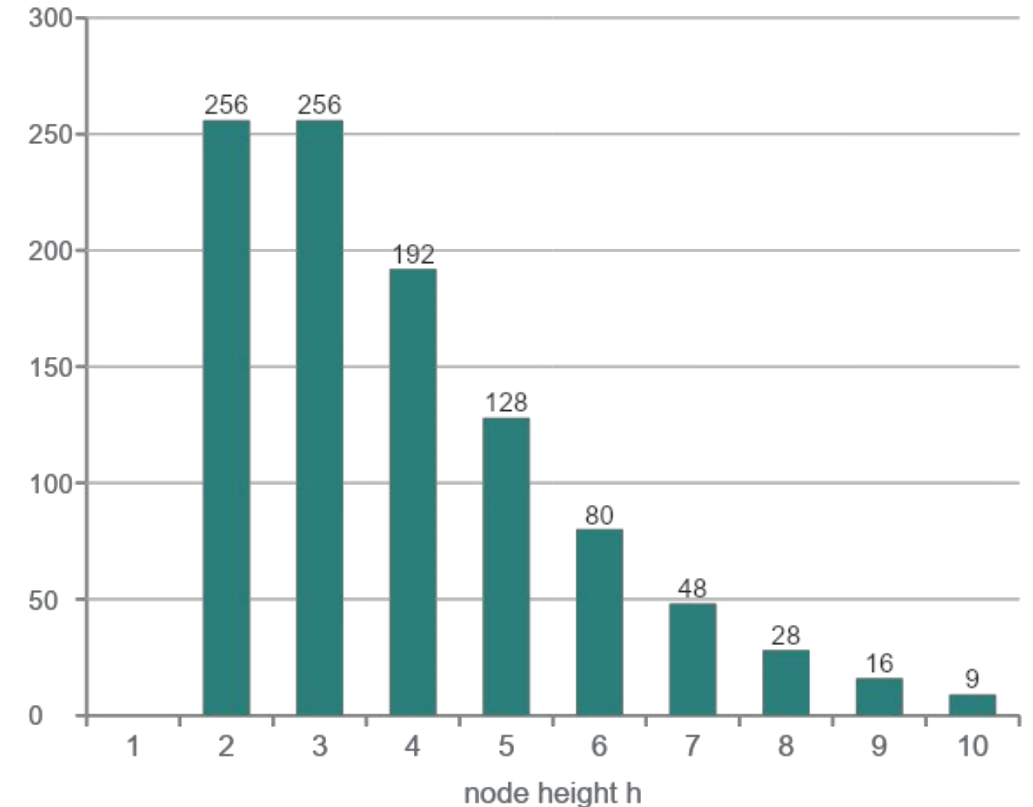
Lemma: $\sum_{i \geq 0} i / 2^i = 2$

$\sum x^i = 1/(1-x)$ for $|x| < 1$. Differentiate: $\sum i x^{i-1} = 1/(1-x)^2$.

Multiply by x : $\sum i x^i = x/(1-x)^2$. Plug in $x = 1/2$: $(1/2)/(1/4) = 2$.

$\Rightarrow$ BUILD-HEAP is **$O(n)$** . Corollary: meld two binary heaps in $O(n)$.

Worst-case sift-down swaps by node height ($n = 1023$)



512 leaves do no work; total = 1013 swaps $\leq 2n$. Most nodes are short!

Scorecard so far

Operation	Unsorted list	Sorted list	Binary heap
insert	$O(1)$	$O(n)$	$O(\log n)$
min	$O(n)$	$O(1)$	$O(1)$
removeMin	$O(n)$	$O(1)$	$O(\log n)$
decreaseKey	$O(1)$	$O(n)$	$O(\log n)$
delete (by handle)	$O(1)$	$O(1)$	$O(\log n)$
meld	$O(1)$	$O(n)$	$O(n)$
build from n items	$O(n)$	$O(n \log n)$	$O(n)$

Takeaways

- Every core operation is $O(\log n)$ — **balanced**, as promised.
- Space: $\Theta(n)$ contiguous storage and no per-node pointers..
- The one weakness: **meld** is linear.
- Part 4 fixes meld; Part 3 caches in on removeMin.



3

PART 3

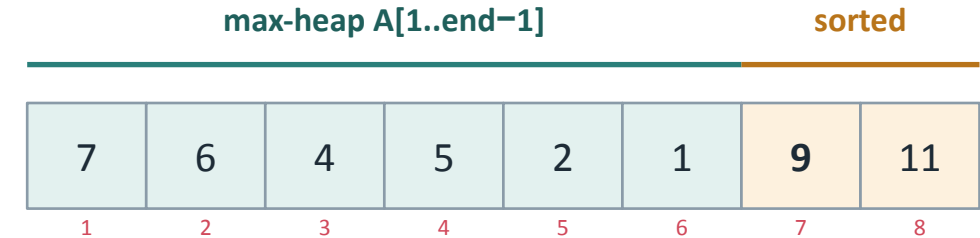
HeapSort

PQ-Sort with a heap — done in place

≈ 13 min

HeapSort: build once, then extract $n - 1$ times

- **Phase 1.** BUILD-MAX-HEAP on the input array — $O(n)$.
- **Phase 2.** Swap the maximum $A[1]$ into its final slot $A[\text{end}]$, shrink the heap by one, and Heapify(1). Repeat.
- The array is a shrinking heap followed by a growing sorted suffix: **no extra memory.**



HEAPSORT (A) :

```

BUILD-MAX-HEAP (A)
for end ← n downto 2:
    swap A[1] ↔ A[end]    ▷ max to final slot
    n ← n - 1             ▷ shrink the heap
    MAX-HEAPIFY (A, 1)
  
```

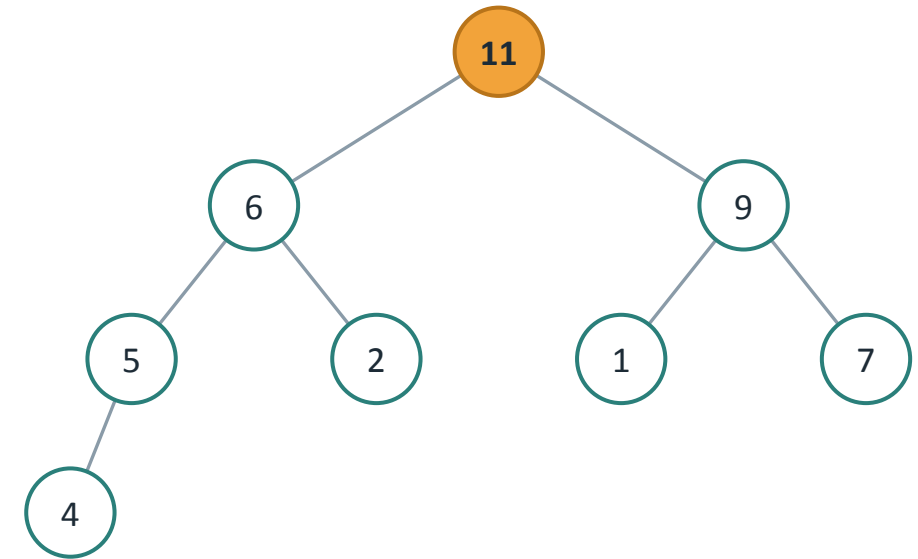
Min-heap or max-heap? A max-heap yields ascending order in place. With the min-heap from our notes, the same loop parks each minimum at the end: **descending** order.

History: J. W. J. Williams introduced the binary heap together with HeapSort (1964); R. W. Floyd gave the $O(n)$ bottom-up construction the same year.

HeapSort trace

input	6	2	9	4	11	1	7	5
BUILD-MAX-HEAP	11	6	9	5	2	1	7	4
end = 8	9	6	7	5	2	1	4	11
end = 7	7	6	4	5	2	1	9	11
end = 6	6	5	4	1	2	7	9	11
end = 5	5	2	4	1	6	7	9	11
end = 4	4	2	1	5	6	7	9	11
end = 3	2	1	4	5	6	7	9	11
end = 2	1	2	4	5	6	7	9	11

teal = heap region; amber = sorted suffix (dark = just placed)



The max-heap after Phase 1

- Each row: one swap + one Heapify(1) on a heap that is one element smaller.
- Students: redo this trace with the min-heap version and confirm you get 11, 9, 7, ..., 1.

Analysis of HeapSort

$O(n \log n)$

WORST CASE

$O(n)$ build + $(n - 1) \cdot O(\log n)$ Heapify

$O(1)$

EXTRA SPACE

in place: heap and output share the array

Not stable

EQUAL KEYS

long-range swaps reorder equal keys

In practice

- Jumps $i \rightarrow 2i$ touch distant memory, so tuned quicksort is usually faster despite HeapSort's better worst case.
- Introsort (used by many C++ `std::sort` implementations) runs quicksort and switches to HeapSort if recursion gets too deep: guaranteed $O(n \log n)$.
- Partial HeapSort gives the k smallest in $O(n + k \log n)$: build once, extract k times.

Sorting algorithms at a glance

Algorithm	Worst	Average	Extra space	Stable	Remark
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	no	PQ-Sort with an unsorted list
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	yes	PQ-Sort with a sorted list; fast on nearly sorted input
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	yes	optimal, but not in place
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$O(\log n)$	no	fastest in practice
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	no	PQ-Sort with a heap, in place

HeapSort is the only one here that is **both** worst-case optimal **and** in place. (at least among the text-book implementations of these algorithms)